

The direct, component-by-component mapping between Architecture Flowchart and n8n Mockup

Phase 1: Ingestion & Routing

This is where the raw data enters and gets categorized.

- **Architecture Diagram:** Jenkins Pre-Prod Pipeline Failure (Trigger)
 - **n8n Canvas Equivalent:** The **Webhook** node.
 - **What it's doing:** It acts as the front door. It waits for PowerShell (or Jenkins) to send the raw JSON payload containing the error log text.
- **Architecture Diagram:** AI Middle Layer Orchestrator
 - **n8n Canvas Equivalent:** The first **Message a model** node + the **Switch** node.
 - **What it's doing:** The Ollama node acts as the "brain," reading the raw text and assigning it a one-word category (like "Security" or "Compliance"). The Switch node acts as the "traffic cop," physically routing the data to different paths based on that word.
- **Architecture Diagram:** Agent 1: Security / Agent 2: Compliance / Uncategorized
 - **n8n Canvas Equivalent:** Outputs **0**, **1**, and **2** on the Switch node.
 - **What it's doing:** In our PoC, we streamlined this. Instead of building massive separate agent pipelines, Outputs 0 and 1 flow directly into the memory check, while Output 2 routes to a standalone Slack: SRE Manual Triage node.

Phase 2: The Memory Check (The Brains of the Operation)

This is where the system decides if it needs a human or if it can fix the issue itself.

- **Architecture Diagram:** Search Historical RAG Repo (The Green Diamond)
 - **n8n Canvas Equivalent:** The **If** node.
 - **What it's doing:** In a production environment, this would be a Vector Database search. In our dummy mockup, the If node fakes this by looking for a specific string (e.g., "AWS Secret"). If it sees it, it pretends it found a match in the database (True). If it doesn't, it assumes it's a brand new issue (False).

Phase 3: Path A (Known Issue - The Fast Track)

This path triggers when the If node returns **True**.

- **Architecture Diagram:** Halt Jenkins Instantly -> Slack: Alert Dev -> Update Existing Jira
 - **n8n Canvas Equivalent:** The **Slack: Alert Developer (Known Fix)** and **Jira: Update**

Existing Ticket nodes (using *Edit Fields*).

- **What it's doing:** Because we don't want to accidentally trigger real PagerDuty or Jira alerts during a demo, these Edit Fields nodes act as visual placeholders. They format the exact text payload that *would* be sent to the Slack and Jira APIs, showing exactly how the instant playbook is delivered.

Phase 4: Path B (New Issue - The Self-Learning Path)

This path triggers when the If node returns **False**.

- **Architecture Diagram:** Agent Executes Root Cause Analysis
 - **n8n Canvas Equivalent:** The second **Message a model** node (renamed to Agent: Execute RCA).
 - **What it's doing:** This is the *real* AI heavy lifting. Llama 3 dynamically reads the unknown error log and writes a custom 3-sentence RCA and mitigation plan completely on the fly.
- **Architecture Diagram:** Create NEW Jira Ticket -> Slack: Ask SRE for Approval
 - **n8n Canvas Equivalent:** The **Jira: Create NEW Ticket** and **Slack: Ask SRE for Approval** nodes (using *Edit Fields*).
 - **What it's doing:** Just like in Path A, these nodes package the Llama 3 RCA output into professional UI mockups. It shows stakeholders what the Jira ticket will look like and simulates the interactive "Approve/Reject" buttons the SRE will see in Slack.
- **Architecture Diagram:** Update RAG & Historical Repo
 - **n8n Canvas Equivalent:** The final **Database: Update RAG Repo & Memory** node.
 - **What it's doing:** It provides the visual conclusion to the workflow. It holds the text confirming that the SRE clicked "Approve" and that the data has been committed to memory, completing the feedback loop.

Explanation the "Dummy" Nodes

"For this demonstration, the routing matrix and AI cognition are 100% live—Llama 3 is actively categorizing and writing the RCAs in real-time. For the final endpoints (Slack, Jira, and the Vector Database), I am using static mock nodes. This prevents us from spamming our live Jira boards during testing, while perfectly illustrating the exact data payloads the system generates."

n8n execution flow

Title: Autonomous Pre-Production Triage & Self-Learning RCA for CI/CD Pipelines

The Problem: In standard DevOps pipelines, when a build fails due to a security vulnerability or compliance violation, it creates a manual bottleneck. SREs and DevOps engineers suffer from massive "alert fatigue" because they have to manually read Jenkins logs, identify the error, write a Root Cause Analysis (RCA), create a Jira ticket, and notify the developer. Worse, they often repeat this manual process for the exact same errors, leading to duplicate tickets and wasted hours.

The Solution: An Agentic AI Middle Layer that autonomously intercepts Jenkins failures, checks a historical knowledge base (RAG) to see if the error is known, instantly halts the pipeline, and provides the exact fix to the developer via Slack—only escalating to a human for brand-new, unseen errors.



